From security signals to reviewed, verified change

A customer workflow for OpenFix

Markus Weldon • Independent architecture proposal and prototype

01

UNDERSTAND

Scope the customer problem

02

PROVE

Validate evidence and behavior

03

FIX

Review the exact change

Start with the customer outcome

Original practice scenario: a multi-tenant billing service with a growing security backlog.

PROBLEM

Where is work stuck?

Detection noise, investigation, ownership, patch creation, or review?

SCOPE

One service, one issue class

Identify the security owner, service owner, approved data, and success criteria.

VALUE

Measure accepted fixes

Compare end-to-end effort, verified outcomes, review burden, and failure rate.

The complete security loop

Evidence, acceptance, verification, and closure are separate decisions.



Keep these decisions separate

A status label should express the evidence and authority actually present.

Candidate	A hypothesis worth investigating
Confirmed	Evidence supports the security finding
Accepted	A reviewer chooses owner, priority, and remediation scope
Verified patch	Agreed checks pass for the exact artifact
Closed	Change controls and post-change evidence support closure

From a signal to an attack path

Hypothetical issue: invoice lookup authenticates a caller but omits tenant ownership.

01

Attacker input

Invoice identifier

→

02

Control failure

Lookup omits tenant predicate

→

03

Supported impact

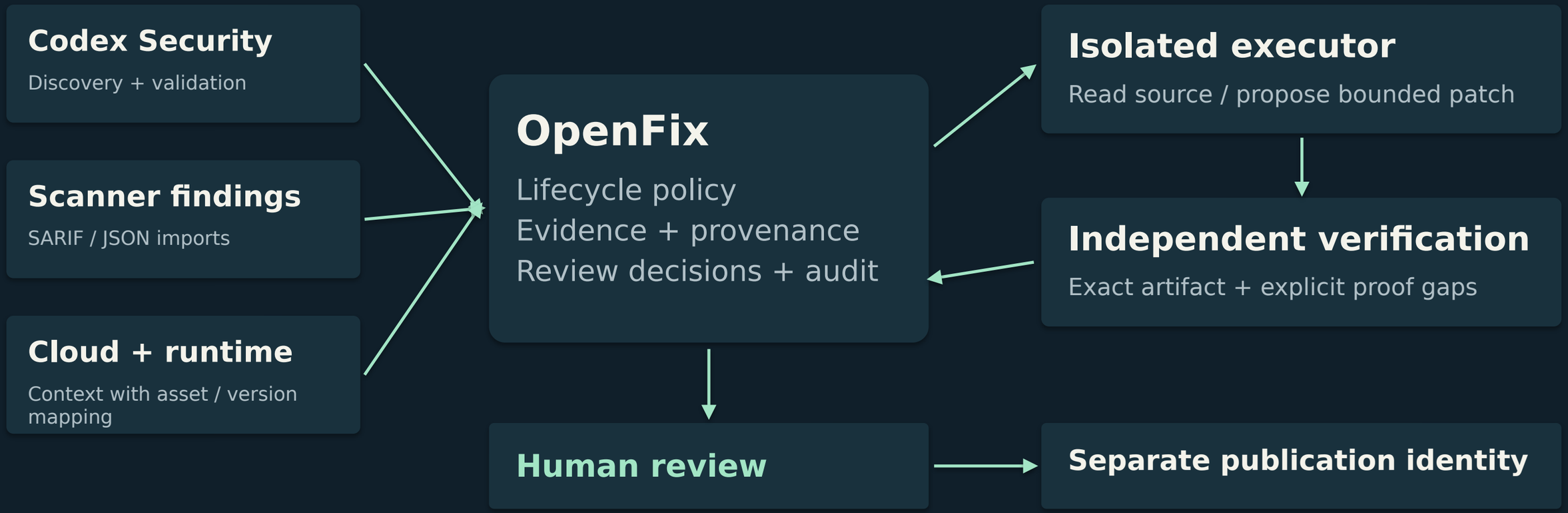
Another tenant invoice is returned

Context informs priority; the code path establishes the flaw.

Cloud exposure: internet-facing asset. Runtime: requests correlated to a known version.
Mapping gaps and unverified exploitation remain visible.

A control plane with clear authority

Target architecture: integrate security capabilities without giving one process every permission.



A useful workflow handles uncertainty

Show the branches as deliberately as the successful fix.

EVIDENCE SUPPORTS IT

Confirmed

Accept for a bounded remediation after reviewing scope and impact.

A CONTROL ALREADY HOLDS

Disproved

Record the evidence and dismissal reason. Do not spend a patch run.

PROOF IS INCOMPLETE

Needs context

Name the missing assumption, test, version mapping, or permission.

One finding, all the way to review

Runnable now: a lifecycle simulation that makes no model or deployment calls.

01	**Create candidate**	Origin, asset, source revision
02	**Record validation**	Method, evidence, limitations
03	**Accept and patch**	Reviewer decision + artifact identity
04	**Verify and review**	Required checks + separate actors
05	**Record change and close**	Rollout + revalidation + explicit reason

Controls travel with the workflow

Assign authority to each action and bind approval to a specific artifact.

Read / investigate	Approved source and data; scoped execution
Propose a patch	Accepted finding; bounded isolated workspace
Verify	Independent result at the exact changed revision
Publish / merge	Separate credentials; reviewer and branch protection
Deploy / close	Normal change process; post-change evidence

A small pilot with a real decision

Agree the baseline, evidence, owners, and exit criteria before expanding.

BASELINE

Current workflow

Approved data, labeled examples, reviewer standards, investigator effort.

EVALUATE

Positive + negative cases

Verified acceptance, false fixes, proof gaps, latency, cost, review time.

DECIDE

Expand, revise, or stop

One service and issue class first. Define the next owner and decision date.

What exists, and what changes

The repository should make implementation status easy to inspect.

CURRENT V1

Remediation orchestrator

SARIF / JSON ingestion, policy routing, Cursor agents, PR CI observation.

TESTED PROTOTYPE

Lifecycle decision gates

Validation, triage, patch identity, verification, review, revalidation, closure.

PROPOSED INTEGRATION

Full customer workflow

Codex Security adapter, isolated jobs, durable evidence, workbench, publication.

Use Pro to learn the workflow

Choose customer deployment from requirements, access, and verified product capabilities.

EXPLORE LOCALLY

ChatGPT sign-in

Use included Codex access and the Security plugin. Verify account access and limits.

INTEGRATE DELIBERATELY

Documented CLI / SDK

Preserve finding artifacts, execution failures, coverage gaps, and exact revision binding.

DEPLOY FOR CUSTOMERS

Explicit operating model

Choose identities, retention, residency, isolation, budgets, and review controls.

Leave with a concrete recommendation

A good customer conversation ends with an owned next step and visible uncertainty.

One service. One issue class. One accountable reviewer.

01

Agree the scope

Problem, assets, data, authority

02

Define the proof

Original issue + legitimate behavior

03

Choose the decision

Success measures + expansion gate

Sources and practical boundaries

Public product documentation supports the workflow; the architecture and scenario are independent.

Scan and evidence review

https://learn.chatgpt.com/docs/security/plugin/scans

Generate, verify, and close a fix

https://learn.chatgpt.com/docs/security/plugin/fix-findings

CLI and SDK contracts

https://learn.chatgpt.com/docs/security/cli/reference

Codex authentication

https://developers.openai.com/codex/auth

ChatGPT and API billing

https://help.openai.com/en/articles/9039756-managing-billing-for-chatgpt-and-the-api-platform